Time-Aware RAG for RBI / SEBI Circulars

A compliance assistant that knows which rule is currently in force

Complete project blueprint: what it is, the problem it solves, how it works, what to build, how to evaluate it, and how to present it in interviews.

Target	SWE roles at large MNCs, with a strong AI/GenAI angle
Duration	About 6 weeks, part-time
Core stack	Python, sentence-transformers, PostgreSQL + pgvector, FastAPI, vLLM (or Ollama / llama.cpp), Docker
Signature idea	Time-aware retrieval that avoids citing superseded regulations
Outcome	A measured, deployable RAG system with numbers you can defend

Contents

1. What the project is

2. The problem it solves

3. How it solves it (the approach)

4. System architecture

5. What you need to build (component by component)

6. Tech stack

7. Data model and corpus

8. Key code starting points

9. Evaluation design

10. Six-week build plan

11. Repository layout

12. Compute, budget, and demo plan

13. Pitfalls to avoid

14. Interview preparation

15. Resume bullets and 60-second pitch

16. Final checklist

1. What the project is

You build a question-answering system over Indian regulatory documents (RBI master directions and circulars, SEBI circulars). A user asks a question such as *"What is the current limit for X under the RBI master direction?"* and gets a streamed answer with numbered citations, the date of each cited circular, and a refusal when the documents do not contain the answer.

Under the hood it is a Retrieval-Augmented Generation (RAG) system in which you personally own every layer:

- **A fine-tuned embedding model** that understands regulatory vocabulary, with the improvement proven on held-out data.
- **A self-hosted open-weights LLM** served with batching and quantization (vLLM on a GPU, or Ollama/llama.cpp without one).
- **A hybrid retrieval layer** (dense vectors + keyword search, fused with RRF) with **time-awareness** built in.
- **A rigorous evaluation harness** and a CI gate so quality cannot silently regress.

Why it stands out. Most student RAG projects call a hosted embedding API and a hosted LLM, then say "it seems to work". This one is measured, benchmarked like a backend service, and built around a real-world failure mode, not a toy demo.

2. The problem it solves

2.1 The general problem

Generic LLMs and naive RAG pipelines have three weaknesses in a compliance setting:

- **Hallucination:** they produce plausible but unsourced rules.
- **Vocabulary gaps:** general embedders handle regulatory phrasing, section numbers and circular IDs imperfectly.
- **No proof of quality:** without a test set, nobody knows whether answers are right.

2.2 The specific problem (your hook): superseded rules

Regulators amend, replace and withdraw circulars constantly. A standard RAG system indexes everything equally, so it can retrieve an **old circular that has since been superseded** and state it confidently as current law. In compliance that is the most damaging kind of error because it looks correct and is sourced.

Your system fixes this by storing validity metadata for every chunk and using it during retrieval, then **measuring** the improvement with a new metric, the **stale-answer rate** (the percentage of questions where the system cites a superseded rule).

The interview story: *"I measured that my baseline cited outdated rules X% of the time and reduced it to Y%."* That is far stronger than "I improved Recall@5" alone.

3. How it solves it (the approach)

Five ideas work together:

Idea	What you do	Why it matters
Time-aware retrieval	Store issue_date, status (active/superseded), supersedes, document_id per chunk. Filter or down-rank superseded chunks in SQL. Answers say "as per circular dated X".	Prevents confident citation of rules no longer in force.
Domain-tuned embedder	Generate synthetic questions per chunk, mine hard negatives, fine-tune a small embedding model with contrastive loss.	Cheap to serve; large relative gain on domain vocabulary.
Hybrid search + RRF	Dense top-k from pgvector plus keyword top-k from Postgres full-text, merged with Reciprocal Rank Fusion.	Exact identifiers (circular numbers, section IDs) are where dense retrieval misses.
Grounded generation	Prompt answers only from numbered context, require citations, refuse when retrieval similarity is low.	Reduces hallucination; compliance answers must be sourced.
Measured everything	Hand-written, document-disjoint test set; retrieval, generation and serving metrics; CI gate.	Turns a demo into engineering evidence.

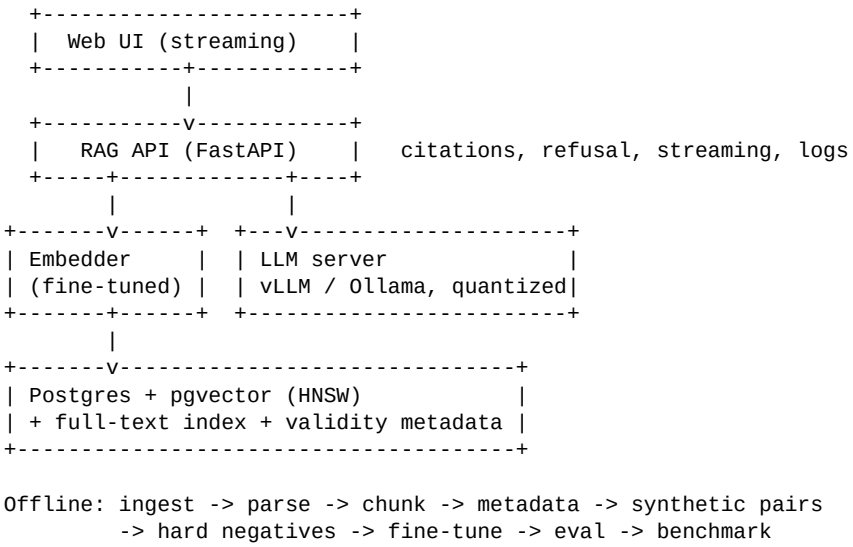
3.1 Request flow (online)

- Client sends a question to `/ask`.
- API embeds the query with your fine-tuned model.
- Retrieval: dense top-k (pgvector) and keyword top-k (Postgres full-text), restricted or re-ranked by `status = active` / date, fused with RRF.
- Optional cross-encoder reranker trims to the best 4 to 6 chunks.
- If the best similarity is below the threshold, return a refusal without calling the LLM.
- Otherwise build a numbered-context prompt; the LLM streams tokens back; the answer cites chunk numbers and circular dates.

3.2 Offline pipeline

Ingest PDFs, parse, chunk by section, extract metadata, generate synthetic pairs, mine hard negatives, fine-tune, evaluate, benchmark.

4. System architecture



Design decisions you should be able to justify

Decision	Choice	Trade-off
Vector store	pgvector (HNSW)	One DB for vectors, metadata, full-text and date filters; a dedicated vector DB wins only at much larger scale.
Embedding model	Small model (bge-small class), fine-tuned	Fast to train and serve; domain tuning gives a big gain per unit cost.
LLM serving	vLLM, quantized (or Ollama/llama.cpp)	Continuous batching and paged KV cache give much higher throughput than a naive generate loop.
Hybrid retrieval	Dense + full-text + RRF	Handles exact circular numbers and section IDs.
Service layout	API + Postgres + LLM server	Deliberately simple. A separate micro-batched embedding service is optional.

5. What you need to build

Everything you build, grouped by component. Items marked **[core]** are required; **[optional]** items are the first to cut if time is short.

Component	What to build	Done when
1. Ingestion [core]	Download RBI/SEBI documents (100 to 300). Parse PDFs (log failures). Clean text, remove headers/footers.	One command rebuilds the index from raw docs.
2. Metadata extraction [core]	Extract issue_date, circular number, title, document_id, and supersession links ("supersedes / in supersession of ..."). Set status.	Every chunk has date + status; sample of 30 manually verified.
3. Chunking [core]	Structure-aware chunking by headings/sections (200 to 400 tokens, 10 to 15 percent overlap), section path prefixed into text.	Chunks preserve section context.
4. Index [core]	Postgres table with embedding vector (HNSW), tsvector column, and metadata columns.	Dense and keyword queries both work.
5. Eval set [core]	50 answerable, 20 unanswerable, 15 to 20 time-trap questions, correct chunks marked, split by document.	One command prints the baseline metric table.
6. Embedder fine-tuning [core]	Synthetic queries, round-trip filter, hard negatives, MultipleNegativesRankingLoss, 4+ configurations.	Results table with at least 4 rows.
7. Temporal + hybrid retrieval [core]	Status/date filter or down-rank, RRF fusion, refusal threshold.	Stale-answer rate measured before and after.
8. Reranker [optional]	Cross-encoder rerank of top candidates.	Gain and added latency reported.
9. RAG API [core]	FastAPI /ask with streaming, numbered citations with dates, refusal path, per-stage timing logs.	Streams an answer with sources.
10. UI [core]	Simple page that streams tokens and lists sources with dates.	Demo works in the browser.
11. Serving benchmark [core, simplified]	One comparison, e.g. fp16 vs 4-bit: TTFT, tokens/s, p95, GPU memory.	One table and one latency chart.
12. Generation eval [core]	Faithfulness, answer correctness, citation accuracy, refusal accuracy.	Numbers in README.
13. CI gate [core]	Test that fails if Recall@5 or MRR@10 drop below baseline minus a tolerance.	Blocks regressions.

Component	What to build	Done when
14. Micro-batching embedding service [optional]	Async queue that batches embed requests.	Max-batch / max-wait benchmark.

What to cut

Redis cache, rate limiting, OpenTelemetry, ONNX export, speculative decoding. They add surface area you cannot explain deeply. A smaller project you fully understand beats a large one you half understand.

6. Tech stack

Layer	Tools	Notes
Parsing	pdfplumber / PyMuPDF, regex	RBI/SEBI PDFs are messy; log parse failures.
Fine-tuning	sentence-transformers, datasets, PyTorch	Free Colab or Kaggle GPU is enough.
Synthetic data	Your own LLM server (OpenAI-compatible)	Saves API cost, shows the serving stack.
Storage	PostgreSQL + pgvector (HNSW) + full-text	Metadata filters live in the same SQL.
LLM serving	vLLM (GPU) or Ollama / llama.cpp (no GPU)	AWQ/GPTQ 4-bit for 16 GB GPUs; GGUF for CPU.
API	FastAPI, asyncpg, httpx	Streaming endpoint.
Evaluation	sentence-transformers evaluators, custom metrics, optional RAGAS	Retrieval metrics fully in your control.
Ops	Docker Compose, GitHub Actions, pytest	CI runs the retrieval gate.

Model names change quickly. Check Hugging Face for a current small embedding model and a current 3B to 8B instruct model with an AWQ/GPTQ or GGUF build, and pin exact versions.

7. Data model and corpus

Record the corpus source, retrieval date and license/terms in your README. Check each regulator's terms before publishing; if unsure, publish scripts and metadata, not the documents.

```
CREATE EXTENSION IF NOT EXISTS vector;
CREATE TABLE chunks (
  id           BIGSERIAL PRIMARY KEY,
  document_id  TEXT NOT NULL,
  circular_no  TEXT,
  title        TEXT,
  section      TEXT,
  text         TEXT NOT NULL,
  issue_date   DATE,
  status       TEXT CHECK (status IN ('active','superseded')) DEFAULT 'active',
  supersedes   TEXT[],           -- document_ids this one replaces
  superseded_by TEXT,
  embedding    vector(384),      -- match your model's dimension
  tsv          tsvector GENERATED ALWAYS AS (to_tsvector('english', text)) STORED
);
CREATE INDEX ON chunks USING hnsw (embedding vector_cosine_ops);
CREATE INDEX ON chunks USING gin (tsv);
```

Time-aware retrieval query (idea)

```
SELECT id, 1 - (embedding <=> $1::vector) AS sim
```

```
FROM chunks
WHERE status = 'active'          -- or: down-rank instead of filtering
ORDER BY embedding <=> $1::vector
LIMIT 20;
```

Filtering is the simplest fix. Also keep a mode that *does* search superseded text, for historical questions such as "what was the rule in 2019?", and compare both modes in your evaluation.

8. Key code starting points

These are working starting points. Adapt names, add error handling, and pin library versions.

8.1 Structure-aware chunking

```
import re
def chunk_markdown(text, url, max_words=250, overlap=40):
    chunks, path = [], []
    for block in re.split(r"(?m)^(?=#{1,4} )", text):
        m = re.match(r"(#{1,4}) (.+)", block)
        if m:
            level = len(m.group(1))
            path = path[: level - 1] + [m.group(2).strip()]
        words = block.split()
        step = max_words - overlap
        for i in range(0, max(len(words), 1), step):
            piece = " ".join(words[i : i + max_words])
            if len(piece.split()) < 30:
                continue
            section = " > ".join(path)
            chunks.append({"text": f"{section}\n{piece}", "url": url, "section": section})
    return chunks
```

8.2 Hard negative mining

```
import numpy as np
def mine_negatives(model, queries, pos_ids, corpus_ids, corpus_texts, doc_of, lo=10, hi=40):
    C = model.encode(corpus_texts, normalize_embeddings=True, batch_size=128)
    Q = model.encode(queries, normalize_embeddings=True, batch_size=128)
    ranked = np.argsort(-(Q @ C.T), axis=1)[:, :hi]
    negs = []
    for qi, row in enumerate(ranked):
        pos = pos_ids[qi]
        ok = [j for j in row[lo:hi]
              if corpus_ids[j] != pos and doc_of[corpus_ids[j]] != doc_of[pos]]
        negs.append(corpus_texts[ok[0]] if ok else None)
    return negs
```

Skipping the top ranks and same-document chunks avoids false negatives (true answers wrongly labeled as wrong). In your domain, also be careful with an amended circular: the old and new versions can both be correct answers to a question about the topic.

8.3 Fine-tuning with contrastive loss

```
from datasets import Dataset
from sentence_transformers import (SentenceTransformer, SentenceTransformerTrainer,
                                   SentenceTransformerTrainingArguments)
from sentence_transformers.losses import MultipleNegativesRankingLoss
from sentence_transformers.training_args import BatchSamplers

model = SentenceTransformer("BAAI/bge-small-en-v1.5")   # or a current small embedder
train_ds = Dataset.from_dict({"anchor": queries, "positive": positives, "negative": negatives})
args = SentenceTransformerTrainingArguments(
    output_dir="runs/ft1", num_train_epochs=3, per_device_train_batch_size=64,
    learning_rate=2e-5, warmup_ratio=0.1, fp16=True,
    batch_sampler=BatchSamplers.NO_DUPLICATES, logging_steps=20, seed=42)
trainer = SentenceTransformerTrainer(model=model, args=args, train_dataset=train_ds,
                                     loss=MultipleNegativesRankingLoss(model))
```

```
trainer.train()
model.save_pretrained("models/bge-small-ft")
```

8.4 Reciprocal Rank Fusion

```
def rrf(*ranked_lists, k=60):
    scores = {}
    for lst in ranked_lists:
        for rank, doc_id in enumerate(lst):
            scores[doc_id] = scores.get(doc_id, 0) + 1 / (k + rank + 1)
    return sorted(scores, key=scores.get, reverse=True)
```

8.5 RAG endpoint (shape)

```
SYSTEM = ("Answer ONLY from the numbered context. Cite sources like [1]. "
          "Mention the circular date for each cited rule. "
          "If the context does not contain the answer, say you do not know.")

@app.post("/ask")
async def ask(q: Question):
    qvec = embed(q.text)
    dense   = await db.fetch(DENSE_SQL, qvec)       # status = 'active'
    lexical = await db.fetch(LEXICAL_SQL, q.text)   # status = 'active'
    ids  = rrf([r["id"] for r in dense], [r["id"] for r in lexical])[:6]
    rows = await fetch_chunks(ids)
    if not rows or best_dense_similarity(dense) < MIN_SIM:
        return {"answer": "I could not find this in the documentation.", "sources": []}
    ctx = "\n\n".join(f"[{i+1}] ({r['circular_no']}, {r['issue_date']})\n{r['text']}"
                      for i, r in enumerate(rows))
    # stream tokens from the OpenAI-compatible LLM server (vLLM / Ollama) ...
```

Choose MIN_SIM from your eval: plot the similarity distribution of answerable vs unanswerable questions, pick the best-separating threshold, and report refusal precision and recall.

9. Evaluation design

This is what separates a serious project from a demo. Report all three layers, and fill tables only with numbers you measured.

9.1 Test sets (write these before any tuning)

Set	Size	Purpose
Answerable questions	~50	Headline retrieval and answer quality. Hand-written, correct chunks marked, document-disjoint from training.
Unanswerable questions	~20	Refusal precision and recall.
Time-trap questions	15 to 20	Old and new circular disagree. Measures stale-answer rate.
Synthetic set	any	Secondary check only; biased toward what the generator finds easy.

9.2 Retrieval table

Configuration	Recall@5	MRR@10	nDCG@10	Stale-answer rate	Latency
Off-the-shelf embedder, dense only	__	__	__	__	__
Fine-tuned, synthetic pairs only	__	__	__	__	__
Fine-tuned + hard negatives	__	__	__	__	__
Fine-tuned + hybrid (full-text + RRF)	__	__	__	__	__
Fine-tuned + hybrid + time-aware filter	__	__	__	__	__

Configuration	Recall@5	MRR@10	nDCG@10	Stale-answer rate	Latency
... + reranker (optional)	__	__	__	__	__

9.3 Generation

- **Faithfulness:** are all claims supported by the retrieved context? Use an LLM judge (name the model, note its bias) and manually audit 20 answers.
- **Answer correctness:** compare to reference answers or key facts you wrote.
- **Citation accuracy:** does the cited chunk actually contain the claim? Partly checkable programmatically.
- **Refusal quality:** correct refusals on unanswerable questions, and wrong refusals on answerable ones.
- **Date correctness:** does the answer cite the in-force circular, with the right date?

9.4 Serving (simplified)

Setup	TTFT p50/p95	Tokens/s	GPU memory	Cost per 1K queries
fp16	__	__	__	__
4-bit quantized	__	__	__	__

Fair benchmarking rules: same prompts, same max_tokens, exclude warm-up, at least 3 runs, report the hardware. Cost per 1K queries = (GPU price per hour / queries per hour at sustained load) x 1000; state your assumptions.

> Be honest about statistics: with 50 test questions a 2-point difference is noise. State the number of queries and avoid over-claiming small gains. Include a 'what did not work' list.

10. Six-week build plan

Week	Focus	Deliverable
1	Corpus and baseline: download 100 to 300 documents, parse PDFs (log failures), chunk by section, extract issue_date and supersession info, answer one question end to end.	Single command rebuilds the index.
2	Evaluation set FIRST: 50 answerable, 20 unanswerable, 15 to 20 time-trap questions; split by document; record baseline Recall@5, MRR@10, nDCG@10, stale-answer rate.	One command prints the baseline table.
3	Fine-tune the embedder: synthetic queries with round-trip filter, hard negatives, MNRL training, at least 4 configurations including one that does not help; general sanity check.	Results table with 4+ rows.
4	Temporal retrieval + hybrid: status/date filtering, RRF, optional reranker; re-run all metrics; one serving comparison (fp16 vs 4-bit).	Filled retrieval table + serving table.
5	The app: FastAPI /ask with streaming, citations with dates, refusal path, simple UI, Docker Compose (API + Postgres + LLM).	docker compose up gives a working demo.
6	Finish and prove it: generation eval, CI gate, error analysis of 20 worst failures, README with 'what did not work', 2-minute demo video, user testing with 5 to 10 people.	Repo, video and links ready.

11. Repository layout

```
rag-compliance/
  README.md              results tables, architecture, what did not work
  docker-compose.yml
  data/                  raw docs (gitignored), processed chunks
  ingest/                fetch_docs.py  parse_pdf.py  chunk.py  extract_metadata.py  build_index.py
  training/              split_by_doc.py  gen_synthetic.py  mine_negatives.py  finetune.py
  eval/                  test_queries.jsonl  unanswerable.jsonl  time_traps.jsonl
                         retrieval_eval.py  generation_eval.py  baseline.json
  api/                   main.py  retrieval.py  prompts.py  Dockerfile
  llm/                   serve.sh  (vLLM or Ollama config)
  bench/                 load_test.py  results/  plot.py
  tests/                 test_retrieval_quality.py  test_api.py
  docs/                  design.md  error_analysis.md
```

12. Compute, budget, and demo plan

- **Fine-tuning:** a small embedder on tens of thousands of pairs trains in minutes to under an hour on free Colab or Kaggle. Save checkpoints to Drive or Hugging Face because sessions disconnect.
- **LLM serving:** a 4-bit quantized 7B-class model fits a 16 GB GPU with modest context; a 3B model is a safer start.
- **Final benchmark:** rent a GPU for a few hours so numbers come from a documented machine.
- **No GPU:** use a GGUF model through llama.cpp or Ollama, keep the fine-tuned embedder (it runs on CPU), and state that in the README.
- **Demo that survives:** keep the LLM URL configurable, record a 2-minute video, and publish the fine-tuned embedder and eval set on Hugging Face (if licensing allows).

13. Pitfalls to avoid

Pitfall	What happens	Fix
Train/test leakage	Test material appears in training; metrics look great and mean nothing.	Split by document before generating anything.
Synthetic-only evaluation	You measure how well the model matches the generator's style.	Hand-written test set is the headline.
False negatives in mining	True answers labeled as negatives; amended circulars are near-duplicates.	Skip top ranks and same-document chunks; inspect samples.
Prefix mismatch	Embedder expects a query instruction; training and serving differ.	Same formatting in training, indexing and serving.
Forgetting to re-embed	Index built with the old model, queries with the new one.	Version the index by model name; rebuild on every change.
Wrong supersession metadata	Time-aware filter hides valid rules or keeps invalid ones.	Manually verify a sample; log extraction confidence.
Messy PDF parsing	Tables and headers corrupt chunks.	Log failures, inspect samples, handle tables separately.
Overfitting small corpus	Gains vanish on new questions.	Fewer epochs; general sanity check.
Unequal benchmarks	One setup looks 10x faster because settings differ.	Identical prompts, max_tokens, hardware, warm-up.
Non-deterministic eval	Metrics move between runs.	Fixed seeds, temperature 0 for judging, multiple runs.

Pitfall	What happens	Fix
Context overflow	Long context silently truncated or OOM.	Limit chunks, set max-model-len, log prompt length.

14. Interview preparation

Q: Why fine-tune the embedder instead of using a bigger general model?

A: A bigger model costs more per query and still lacks your domain's exact vocabulary. A small tuned model is cheaper to serve and on my held-out set improved Recall@5 from __ to __. I measured both.

Q: How does MultipleNegativesRankingLoss work?

A: For each (query, positive) pair the other positives in the batch act as negatives; cross-entropy over similarity scores makes the true pair rank first. Larger batches mean more negatives. I added mined hard negatives and a no-duplicates sampler.

Q: What is the stale-answer problem and how did you solve it?

A: Regulations get superseded, and naive RAG can cite an outdated rule. I stored status and dates per chunk, filtered/down-ranked superseded text, and measured stale-answer rate on hand-written trap questions: __% to __%.

Q: How do you know the improvement is real?

A: Hand-written test set, document-disjoint from training, number of queries reported, ablations, and a general sanity check.

Q: Why hybrid retrieval?

A: Dense retrieval misses exact identifiers like circular numbers and section IDs; full-text catches them; RRF merges rankings without tuning score scales.

Q: Why is vLLM faster than a plain generate loop?

A: Continuous batching lets new requests join the running batch every decoding step, and PagedAttention manages the KV cache in blocks, reducing fragmentation so more sequences fit.

Q: What does quantization cost you?

A: 4-bit weights cut memory and often raise throughput with some quality loss; I measured quality on my own eval instead of assuming.

Q: What dominates latency: prefill or decode?

A: TTFT is mostly prefill (compute-bound, grows with prompt length). Per-token speed afterwards is mostly decode (memory-bandwidth-bound).

Q: How do you reduce hallucination?

A: Grounded prompt, required citations, a similarity threshold with a refusal path, and evaluation of faithfulness and refusal accuracy.

Q: How would you scale it?

A: Stateless API replicas behind a load balancer, LLM servers scaled independently, tuned HNSW or read replicas, caching hot queries, and queued ingestion workers.

15. Resume bullets and 60-second pitch

Replace blanks with numbers you measured. Never publish a number you cannot reproduce from your repo.

- Built a time-aware RAG system over RBI/SEBI circulars; reduced stale-rule citations from __% to __% using validity metadata in hybrid retrieval, evaluated on __ hand-written trap questions.
- Fine-tuned a domain embedding model (synthetic queries, mined hard negatives, contrastive loss), improving Recall@5 from __ to __ and MRR@10 from __ to __ on a hand-written, document-disjoint test set of __ questions.
- Served a quantized open LLM and built a streaming RAG API with pgvector, full-text search and RRF, citations with circular dates, refusal handling, and a CI gate that blocks retrieval regressions.

60-second pitch

"I wanted to understand RAG beyond calling an API, so I built the full stack on RBI and SEBI circulars. The key problem is that regulations get superseded, and a naive RAG system confidently cites outdated rules. I measured that: my baseline cited a superseded rule __% of the time. I fixed it with validity metadata and time-aware hybrid retrieval and brought it to __%. I fine-tuned a small embedder with synthetic queries and hard negatives, improving Recall@5 from __ to __ on a hand-written test set, served an open LLM, and added a CI gate that fails if retrieval quality drops. I also documented what did not work, such as __."

Four things that make it more impactful

- **Real users:** 5 to 10 testers, log questions and thumbs up/down, report "tested with N users".
- **Error analysis:** categorize your 20 worst failures (parsing error, wrong chunk, stale rule, LLM ignored context).
- **Published artifacts:** fine-tuned embedder and eval set on Hugging Face, clean GitHub repo.
- **Honest reporting:** include ablations and a 'what did not work' section.

Optional second hook (pick at most one)

Hindi/Hinglish queries with a multilingual embedder, measuring cross-lingual retrieval; or a fine-tuned cross-encoder reranker with measured gain and latency.

16. Final checklist

Item	Done?
Hand-written test set, never used in training, split by document	[ ]
Time-trap questions and stale-answer rate measured before and after	[ ]
Baseline and fine-tuned metrics in a table with number of queries stated	[ ]
At least 3 ablations including one that did not help	[ ]
Serving benchmark with identical settings across setups	[ ]
Supersession metadata manually verified on a sample	[ ]
docker compose up works from a clean clone	[ ]
CI runs tests and the retrieval gate	[ ]
README with architecture diagram, results, limitations, corpus terms	[ ]
Error analysis of 20 worst failures	[ ]

Item	Done?
2-minute demo video and links on resume	[ ]
You can explain every number and every line of code without notes	[ ]